QntBank

Báo cáo phát triển ứng dụng doanh nghiệp

Thành viên:
Lê Minh Quân - 23020563
Bùi Mạnh Nam - 23020556
Chu Anh Trường - 23020577

Nội dung

1. Giới thiệu về Ứng dụng
2. Các công nghệ sử dụng
3. Devops
4. Demo ứng dụng

Giới thiệu ứng dụng

Công nghệ sử dụng

spring boot

Spring Data JPA

Tailwind CSS

Maven™

spring SECURITY

MySQL

POSTMAN

Swagger

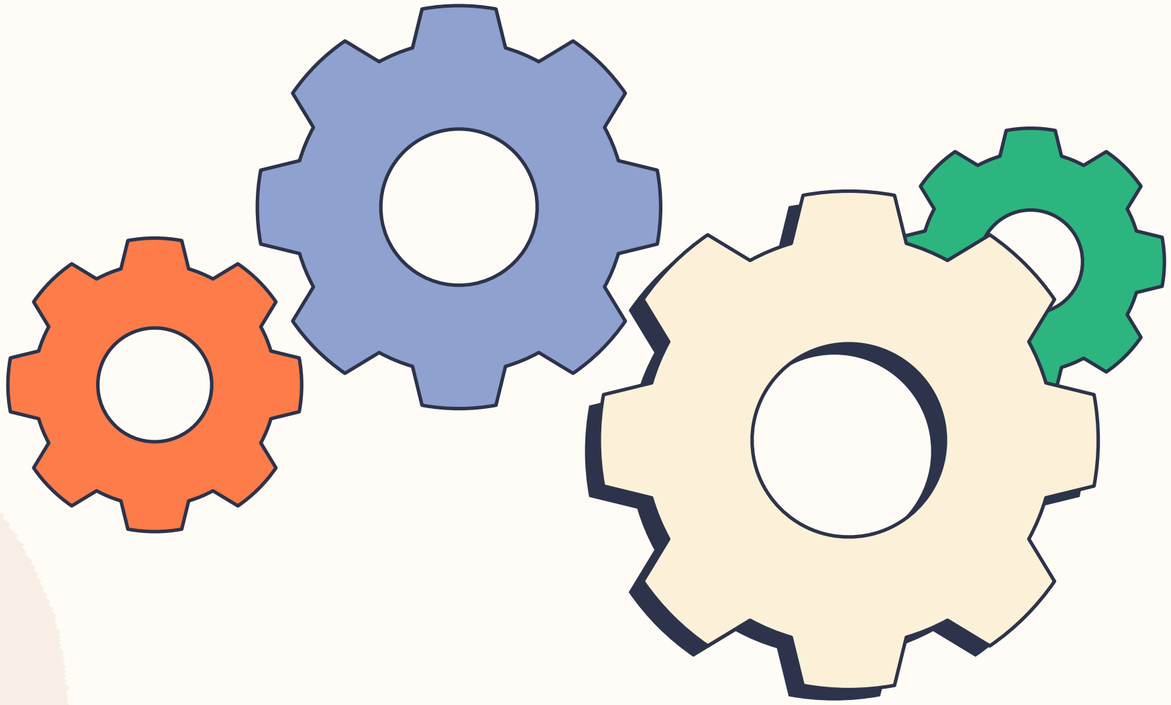
Tổng quan ứng dụng

QntBank là một hệ thống Cổng thông tin ngân hàng số, được xây dựng bởi **SpringBoot Framework** và **Java**

Đặc điểm

- **Bảo mật**
- **Tốc độ**
- **Khả năng chịu tải và mở rộng**
- **Triển khai và cập nhật**
- **Tính sẵn sàng và Chịu lỗi**

Các chức năng của ứng dụng



Các chức năng của ứng dụng

- **Chuyển tiền**

  Cho phép người dùng di chuyển tiền giữa các tài khoản một cách tức thời và an toàn. Hệ thống xử lý kiểm tra số dư, xác thực người nhận (qua tên/số tài khoản), và cập nhật số dư của cả người gửi và người nhận

- **Gửi tiền**

  Cho phép người dùng tăng số dư tài khoản bằng cách nạp tiền vào. Khi kích hoạt, hệ thống sẽ cập nhật lại số dư của người dùng.

- **Rút tiền**

  Cho phép người dùng giảm số dư tài khoản của mình, thường liên kết với việc rút tiền mặt hoặc thanh toán qua thiết bị ngoại vi.Xử lý yêu cầu rút tiền bằng cách kiểm tra nghiêm ngặt giới hạn giao dịch (transaction limits) và đảm bảo tài khoản có đủ số dư

- **Quản lý chi tiêu**

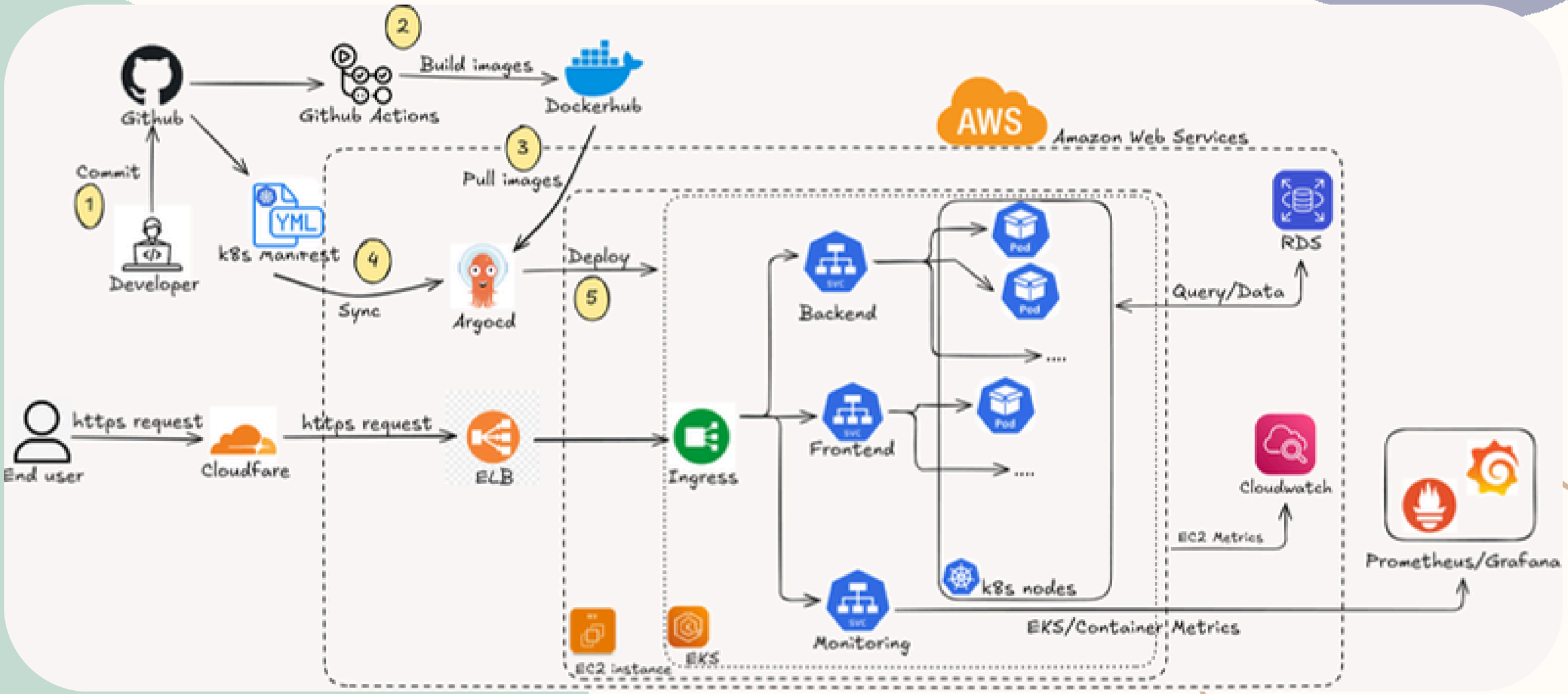
  Cho phép người dùng kiểm soát chi tiêu của mình. Khi thực hiện các chứ năng trên, hệ thống sẽ cập nhật lịch sử giao dịch và trực quan hóa lịch sử thông qua biểu đồ

Phát triển trong tương lai

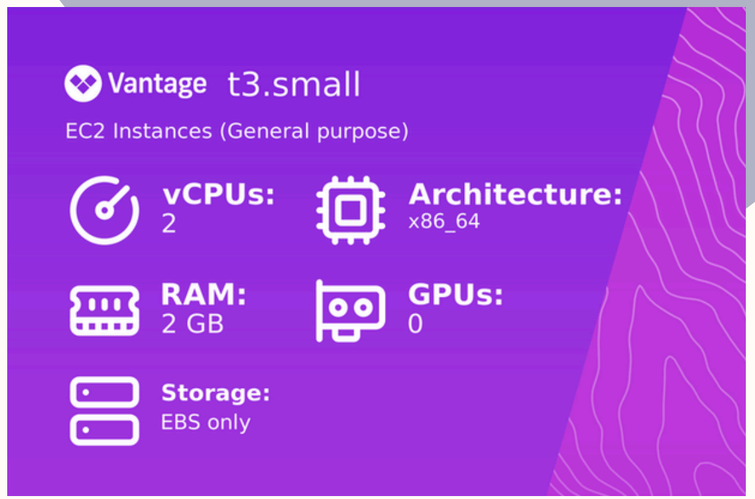
- Tích hợp chatbot để gợi ý người dùng chi tiêu sao cho phù hợp
- Mở thêm các loại tài khoản (tiết kiệm, tín dụng, ...)
- Tách thành các microservices
- Ứng dụng AI giúp phát hiện các giao dịch gian lận

Devops

Pipeline


Github
Commit
1
Developer
2
Github Actions
Build images
Dockerhub
3
Pull images
k8s manifest
YML
4
Sync
Argocd
Deploy
5
AWS
Amazon Web Services
Backend
Frontend
Monitoring
Pod
....
k8s nodes
RDS
Query/Data
Cloudwatch
EC2 Metrics
Prometheus/Grafana
EKS/Container Metrics
End user
https request
Cloudfare
https request
ELB
Ingress
EC2 instance
EKS

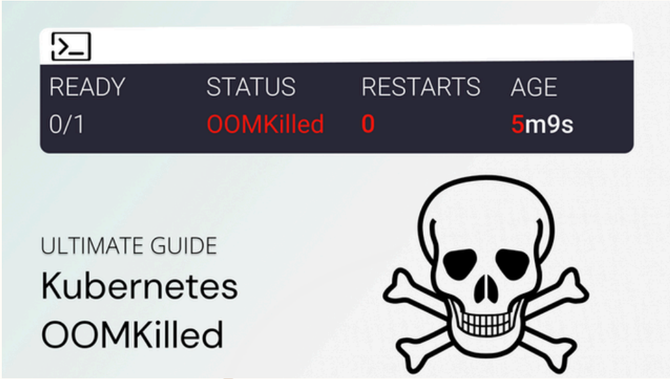
Cấu hình/Tài nguyên Cloud



Sử dụng 4 node t3.small → tổng tài nguyên 8 GB RAM và 8vCPU
Ở trạng thái chưa có tải thì với các thành phần:

- ArgoCD (~ 300MB)
- Prometheus/Grafana (~ 1.1 GB)
- OS, k8s,... (~ 1.3 GB | trên cả 4 node)
- BE + FE (~ 550 MB | 1 pod)

→ Tổng ban đầu đầu đã chiếm ~ 3 GB, còn ~ 5GB.



Mặc dù ~ 5 GB là đủ lớn để scale, nhưng trong môi trường phân tán như k8s có **phân mảnh tài nguyên giữa các node**.
→ Để đảm bảo tính ổn định và tránh lỗi OOMKilled (Out of Memory) hoặc Pending Pod do thiếu tài nguyên cục bộ trên node, nên giới hạn quy mô test tải ở 3 pods BE.

Kịch bản test tải

Công cụ: JMeter (giả lập 100 threads ~ 100 người dùng request liên tục vào hệ thống).
Mỗi pod được giới hạn ở mức 50% CPU

Mục tiêu: Đảm bảo với 3 Pods BE chạy trên hạ tầng phân mảnh, hệ thống vẫn cân bằng tải tốt (qua Service/Ingress) và không có Pod nào bị crash do thiếu tài nguyên.

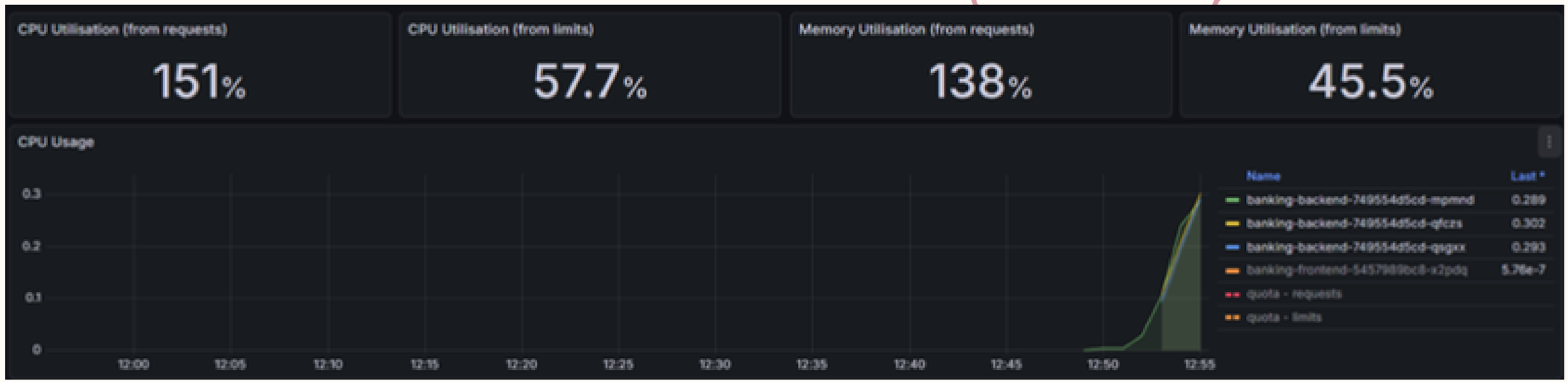
Trước

```
(base) PS C:\Users\elect> kubectl get pods -n deployment
NAME                                  READY   STATUS    RESTARTS   AGE
banking-backend-749554d5cd-mpmnd      1/1     Running   0          51m
banking-frontend-5457989bc8-x2pdq     1/1     Running   0          51m
```

Sau

```
NAME                                  READY   STATUS    RESTARTS   AGE
banking-backend-749554d5cd-mpmnd      1/1     Running   0          81m
banking-backend-749554d5cd-qfczs      1/1     Running   0          3m25s
banking-backend-749554d5cd-qsgxx      1/1     Running   0          3m25s
banking-frontend-5457989bc8-x2pdq     1/1     Running   0          81m
```

các pod chịu tốt, chưa restart




```
NAME                                    CPU(cores)   MEMORY(bytes)
banking-backend-749554d5cd-mpmnd        271m         380Mi
banking-backend-749554d5cd-qfczs        339m         366Mi
banking-backend-749554d5cd-qsgxx        499m         359Mi
banking-frontend-5457989bc8-x2pdq       0m           3Mi
```

Tài nguyên tiêu thụ

DEMO

Thank you for listening!